

UDEM

UNIVERSIDAD DE MONTERREY

Práctica 9

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo
García

Mat: 612348

Nombre: Ricardo Garza
Benítez

Mat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 16 de junio del 2026

Introducción

En el desarrollo de sistemas embebidos con microcontroladores, la medición y presentación de variables físicas en tiempo real representa uno de los escenarios de aplicación más comunes y demandantes. La Práctica 9 integra tres periféricos fundamentales del PIC16F887: el módulo de conversión analógico-digital (ADC), el módulo Timer0 como base de tiempo y la pantalla LCD de 16x2 como interfaz visual de usuario. El resultado es un sistema capaz de mostrar simultáneamente el voltaje medido en un potenciómetro y el tiempo transcurrido desde el encendido.

La práctica se divide en dos actividades progresivas. En la primera se construye un cronómetro básico apoyado en interrupciones de Timer0, que cuenta segundos y los presenta en pantalla en formato MM:SS. En la segunda actividad se incorpora el módulo ADC para leer un nivel de tensión variable y mostrar ambos datos, voltaje y tiempo, en las dos líneas del LCD de manera simultánea. Ambas actividades comparten la misma configuración de oscilador a 16 MHz con cristal externo HS, lo que garantiza una base de tiempo precisa tanto para el cronómetro como para las conversiones analógicas.

Integrar correctamente estos tres periféricos exige comprender cómo conviven las interrupciones de Timer0 con el ciclo principal de polling del ADC, cómo se calcula el valor de voltaje a partir del resultado digital de 10 bits, y cómo se formatea la salida para que el LCD la muestre sin artefactos visuales. Este reporte documenta el diseño, la simulación en Proteus y los resultados obtenidos en la implementación física sobre protoboard.

Marco Teórico

Los conceptos teóricos que fundamentan esta práctica abarcan el funcionamiento del Timer0, el módulo ADC de 10 bits y el manejo del LCD mediante biblioteca de funciones en C.

Timer0 e Interrupciones

El Timer0 del PIC16F887 es un temporizador/contador de 8 bits gobernado por el registro TMR0. Su comportamiento se controla a través del registro OPTION_REG, donde los bits PS2:PS0 definen el divisor de frecuencia (prescaler). Cuando el registro TMR0 desborda desde 0xFF a 0x00, se activa la bandera T0IF del registro INTCON, y si el bit T0IE está habilitado junto con el habilitador global GIE, se ejecuta la rutina de servicio de interrupción ISR. En la práctica se configuró OPTION_REG = 0x07, lo que selecciona un prescaler de 1:256 asignado al Timer0; con un cristal de 16 MHz la frecuencia de instrucción es de 4 MHz y cada ciclo de instrucción dura 0.25 μ s, por lo que el Timer0 desborda aproximadamente cada 38.4 ms con valor inicial de 100. Acumulando 100 desbordamientos se obtiene aproximadamente un segundo, valor que se usa para incrementar el contador de tiempo en la variable 'tiempo' [1].

Módulo ADC (Convertidor Analógico-Digital)

El PIC16F887 incorpora un módulo ADC de 10 bits con hasta 14 canales analógicos seleccionables. La conversión transforma un voltaje de entrada entre VSS y VREF+ en

un valor digital de 10 bits almacenado en los registros ADRESH:ADRESL. El registro ADCON0 controla la selección de canal, el inicio de conversión mediante el bit GO/DONE y la habilitación del módulo con el bit ADON. El registro ADCON1 define la justificación del resultado (derecha con ADFM=1) y la configuración de las referencias de voltaje. En la práctica se usó el canal AN0 (pin RA0) con referencia positiva de 5 V, por lo que la conversión del valor digital a voltaje sigue la relación: $\text{Voltaje} = (\text{ADC_raw} \times 5000 \text{ mV}) / 1023$. El resultado se divide en parte entera y fraccionaria para su presentación en el LCD [1] [2].

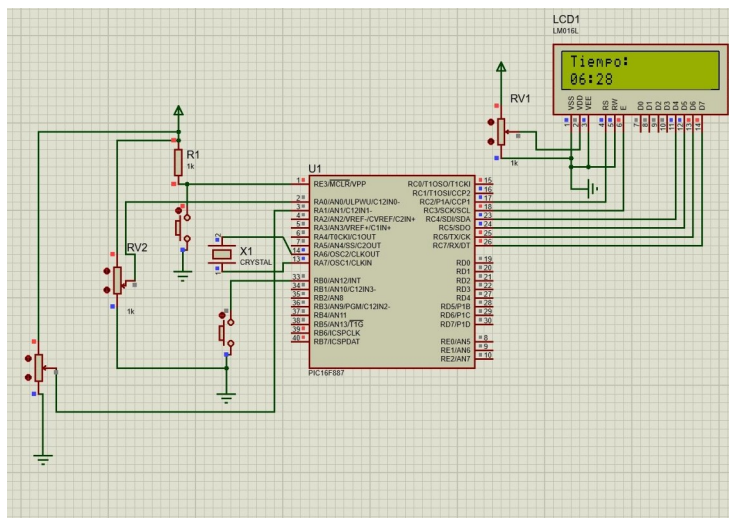
Pantalla LCD 16x2 (HD44780)

La pantalla LCD LM016L utilizada en Proteus es compatible con el controlador HD44780, estándar de la industria para displays alfanuméricos. El módulo opera en modo de 4 u 8 bits de datos; en esta práctica se emplearon los pines D4 a D7 del controlador conectados al PORTC del PIC, junto con las señales RS, E y RW. La función `LCD_Set_Cursor(fila, columna)` posiciona el cursor en la coordenada deseada del display de 16 columnas por 2 filas, y `LCD_puts()` escribe una cadena de caracteres. Para evitar parpadeos y caracteres residuales, la cadena formateada con `sprintf()` se construye con longitud fija de 16 caracteres, relleno con espacios al final [2] [3].

Formato de tiempo y uso de `sprintf()`

La función `sprintf()` de la biblioteca estándar de C permite construir cadenas con formato numérico controlado. En la práctica se usa el especificador `%02u` para garantizar que los valores de minutos y segundos siempre se muestren con dos dígitos, agregando un cero a la izquierda cuando el valor es menor a 10. La expresión `tiempo/60` entrega los minutos y `tiempo%60` los segundos, ambas operaciones sobre enteros sin signo de 16 bits para evitar desbordamiento en tiempos prolongados [3].

Simulación — Actividad 1: Cronómetro con Timer0



Simulación — Actividad 2: Voltímetro y Cronómetro

Actividad 1 — Cronómetro con Timer0

En la primera actividad se implementó un cronómetro que utiliza el Timer0 del PIC16F887 operando bajo interrupciones. El sistema mostró en la primera línea del LCD el texto estático 'Tiempo:' y en la segunda línea el conteo actualizado en formato MM:SS. Durante las pruebas en Proteus se verificó que el tiempo avanzaba de manera continua y sin saltos, lo cual confirma que la acumulación de 100 desbordamientos de Timer0 con prescaler 1:256 produce una base de tiempo de aproximadamente un segundo con el cristal de 16 MHz. La segunda línea del display se actualizaba con la frecuencia del ciclo principal del while, sin generar parpadeo perceptible en la simulación.

En la implementación física sobre protoboard el cronómetro funcionó correctamente desde el primer encendido. El LCD desplegó el conteo en tiempo real y el formato MM:SS se mantuvo estable incluso al superar el minuto, donde la pantalla pasó de '00:59' a '01:00' sin errores de visualización. No se presentaron problemas de rebote ni de pérdida de interrupciones durante el tiempo de prueba.

Actividad 2 — Voltímetro con ADC y Cronómetro simultáneo

En la segunda actividad se integró el módulo ADC para medir el voltaje en el pin AN0 a través de un potenciómetro, mientras que el Timer0 continuó generando la base de tiempo de manera paralela mediante interrupciones. La primera línea del LCD mostró el voltaje medido con formato 'Voltaje: X.XXV' y la segunda línea el tiempo acumulado en formato MM:SS posicionado en la columna 10. Al girar el potenciómetro durante las pruebas, el valor de voltaje en pantalla respondió de manera inmediata y proporcional, confirmando el correcto funcionamiento de la cadena de conversión: lectura del ADC de 10 bits, escalado a milivoltios mediante la relación $(\text{raw} \times 5000) / 1023$ y separación en parte entera y fraccionaria para su presentación.

En el circuito físico se observó que el voltaje mostrado en el LCD coincidió con la lectura de un multímetro colocado en el mismo nodo, con una diferencia inferior a 50 mV atribuible a la tolerancia del potenciómetro y a la resolución de 4.88 mV por bit del ADC de 10 bits con referencia de 5 V. El cronómetro continuó avanzando correctamente durante toda la prueba, sin que la rutina de polling del ADC y el delay de 200 ms en el ciclo principal interfirieran de manera apreciable con la base de tiempo generada por las interrupciones.

Conclusiones Individuales

Ricardo Garza:

Lo que más me llamó la atención de esta práctica fue ver cómo el Timer0 con interrupciones y el ADC con polling coexisten sin pisarse. En teoría parecía sencillo: la ISR maneja el tiempo y el main lee el voltaje, pero en la implementación física noté que el delay de 200 ms del ciclo principal hacía que el cronómetro se atrasara ligeramente si el prescaler no estaba bien ajustado. El detalle de cargar TMR0 = 100 dentro de la ISR fue clave para compensar los ciclos consumidos por la propia interrupción y que la acumulación de 100 desbordamientos diera efectivamente un segundo. También tuve que depurar el formato del printf() porque al principio la cadena de voltaje no tenía longitud fija y dejaba caracteres residuales en el LCD cuando el número bajaba de un

dígito entero; rellenar con espacios al final solucionó el problema. En general, esta práctica me dejó claro que en sistemas con múltiples periféricos activos, la sincronización y el control preciso de los tiempos de ejecución son tan importantes como la lógica de la aplicación en sí.

Eugenio Romo:

Esta práctica me permitió comprender de forma clara cómo un microcontrolador puede manejar dos tareas aparentemente independientes, la medición de voltaje y el conteo de tiempo, mediante la separación entre el flujo principal del programa y las interrupciones. El módulo ADC convierte una señal analógica continua en un valor discreto de 10 bits, y entender la fórmula de escala hacia milivoltios me ayudó a dimensionar la resolución real del sistema: con 5 V de referencia y 1024 niveles, cada paso equivale a poco menos de 5 mV. Por su parte, el Timer0 actúa como un marcapasos del sistema; su configuración correcta es lo que garantiza que el tiempo desplegado sea confiable. La práctica dejó en evidencia que conocer los periféricos a nivel de registros, y no solo a través de funciones de biblioteca, es fundamental para poder ajustar el comportamiento del microcontrolador a los requisitos reales de cada aplicación.

Referencias

[1] Microchip Technology. (2009). PIC16F887 Data Sheet: 28/40/44-Pin Enhanced Flash Microcontrollers. Microchip Technology Inc.

<https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>

[2] Hitachi Semiconductor. (1998). HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver. Hitachi Ltd.

<https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>

[3] Merino Pérez, L. A. (2012). Microcontroladores PIC: Diseño práctico de aplicaciones (2.^a ed.). McGraw-Hill.